INTERVIEW DOCS

Backend System Design

Scalability, Databases, Caching, and Designing for Interviews

Java Agentic AI — Knowledge Base Reference

1. How to Approach a System Design Interview

- 1. Clarify requirements — functional (what must it do) and non-functional (scale, latency, consistency needs). Never assume; ask.
- 2. Estimate scale — back-of-envelope numbers for users, requests/second, storage, bandwidth.
- 3. Define the API — key endpoints/operations the system exposes.
- 4. High-level design — draw the major components and how data flows between them.
- 5. Deep dive — pick the 1-2 hardest sub-problems (e.g. the data model, or handling a hot key) and go deep with trade-offs.
- 6. Identify bottlenecks and discuss scaling, failure handling, and monitoring.

2. Scalability Fundamentals

Concept	Meaning
Vertical scaling	Adding more power (CPU/RAM) to a single machine — simple, but has a hard ceiling
Horizontal scaling	Adding more machines and distributing load — more complex, but scales further
Load balancer	Distributes incoming traffic across multiple server instances (round-robin, least-connections, etc.)
Stateless services	Servers hold no client-specific state, so any instance can serve any request — a prerequisite for easy hori

3. Caching

Caching stores frequently-accessed data in a faster layer (in-memory) to reduce load on the primary database and cut latency.

Strategy	How it works	Trade-off
Cache-aside (lazy loading)	App checks cache first; on miss, reads DB and populates cache	Simple, but first request always misses (cold cache
Write-through	Every write goes to cache and DB together	Cache always fresh, but adds write latency
Write-behind	Write to cache immediately, DB updated asynchronously later	Fast writes, but risk of data loss if the cache crash

- Eviction policies: LRU (Least Recently Used) is most common; also LFU, FIFO, TTL-based expiry.
- Common tools: Redis, Memcached; CDNs cache static assets close to end users geographically.
- Cache invalidation is famously one of the hardest problems — stale cache data can silently cause incorrect behavior long after the underlying data changed.

4. Database Scaling

4.1 Replication

A primary (master) database handles writes; one or more replicas (read-only copies, kept in sync asynchronously or synchronously) handle read traffic, distributing read load and providing failover if the primary goes down.

4.2 Sharding (Horizontal Partitioning)

Splits a large dataset across multiple database instances by some key (e.g. user ID range, hash of user ID, geographic region), so each shard only holds a fraction of the total data — necessary when a single machine can no longer hold or serve the entire dataset.

Sharding strategy	Description	Risk
Range-based	Partition by a value range (e.g. user IDs 1-1M, 1M-2M)	Uneven distribution if some ranges are hotter
Hash-based	Partition by hash(key) mod N	Resharding when N changes requires moving a lot
Directory-based	A lookup service maps each key to its shard	The lookup service itself becomes a bottleneck/sin

4.3 SQL vs NoSQL

Aspect	SQL (Relational)	NoSQL
Schema	Fixed, strongly typed schema	Flexible/schema-less (document, key-value, wide-column, graph)
Consistency	Strong (ACID transactions)	Often eventual consistency (tunable in some systems)
Scaling	Traditionally vertical, sharding is harder	Designed for horizontal scaling from the start
Use case	Complex queries, joins, strong consistency needs (financial, inventory)	Massive scale, flexible schema, high write throughput (activity feeds, c
Examples	PostgreSQL, MySQL	MongoDB, Cassandra, DynamoDB, Redis

5. CAP Theorem

In a distributed system, during a network partition you must choose between **Consistency** (every read sees the latest write) and **Availability** (every request gets a response, even if not the latest data) — you cannot have both simultaneously alongside Partition tolerance, which is effectively mandatory for any distributed system. Systems are typically described as CP (favors consistency, e.g. traditional RDBMS with strict replication) or AP (favors availability, e.g. Cassandra, DynamoDB).

6. Message Queues & Asynchronous Processing

Queues (Kafka, RabbitMQ, SQS) decouple producers from consumers, absorb traffic spikes, and enable asynchronous, resilient processing — a producer can keep accepting requests even if a downstream consumer is temporarily slow or down.

Pattern	Description
Point-to-point queue	Each message consumed by exactly one consumer (competing consumers for scaling)
Pub/Sub	Each message broadcast to every subscriber of a topic
Dead-letter queue	Failed/unprocessable messages routed aside for inspection instead of blocking the queue

7. Consistent Hashing

Used to distribute keys across a changing set of nodes (cache servers, shards) while minimizing redistribution when nodes are added/removed — nodes and keys are mapped onto a hash ring, and each key is owned by the next node clockwise on the ring, so adding/removing one node only remaps the keys immediately adjacent to it rather than the entire keyspace.

8. Rate Limiting

Algorithm	Idea
Fixed window counter	Count requests in a fixed time window (e.g. per minute); simple but can allow bursts at window boundar
Sliding window log	Track precise timestamps of recent requests; accurate but more memory-intensive
Token bucket	Tokens added at a steady rate; a request consumes a token, allowing controlled bursts up to bucket ca
Leaky bucket	Requests processed at a fixed steady rate regardless of burstiness, smoothing output traffic

9. Common System Design Interview Prompts

- Design a URL shortener — key ideas: base62 encoding of an auto-incrementing ID or hash, DB for mapping storage, cache for hot redirects, analytics via async event logging.
- Design a rate limiter — token bucket or sliding window, distributed via Redis for multi-instance consistency.
- Design a notification system — fan-out via message queue, per-channel worker consumers (email/SMS/push), retry with backoff.
- Design a distributed cache — consistent hashing for node assignment, replication for fault tolerance, LRU eviction.
- Design an e-commerce order system — inventory reservation, payment integration, Saga pattern for cross-service consistency, idempotency keys for retried payment requests.

10. Quick Interview Recap

- What's the difference between horizontal and vertical scaling? Vertical adds resources to one machine (limited ceiling); horizontal adds more machines (near-limitless, but adds distributed systems complexity).
- Why is cache invalidation hard? Because there's no single global signal for 'this data changed' across every cache layer/replica, and reasoning about which cached copies are stale under concurrent writes is inherently tricky.
- CP vs AP — pick one example each. CP: a strongly consistent RDBMS cluster. AP: DynamoDB/Cassandra under eventual consistency settings.
- Why use consistent hashing over simple hash(key) % N? Because when N (number of nodes) changes, plain modulo hashing remaps almost every key; consistent hashing only remaps keys near the changed node.